AgroSearch — Motor de Busca Inteligente (TF-IDF implementado do zero)

Relatório · Laboratório Prático 04 — Tópicos Avançados: Recuperação de Informação / PLN · UNIPÊ

1. Arquitetura

A solução está em um único arquivo, `agrosearch_app.py`. O pipeline, o índice, o TF-IDF e o cosseno usam **apenas a biblioteca padrão do Python** (`re`, `unicodedata`, `math`, `collections.Counter`). Nenhuma biblioteca de PLN ou de aprendizado de máquina é importada. Streamlit e pandas servem só para a interface e para exibir tabelas. A interface tem quatro abas, uma por fase, e uma barra lateral com as caixas de seleção de **stopwords** e **stemming**, o esquema de TF e a base de documentos editável; qualquer mudança reconstrói o índice na hora.

2. Fase 1 — Pipeline de pré-processamento

(1) Tokenização por expressão regular (sequências de letras, incluindo acentuadas, e dígitos); **(2) normalização**: minúsculas e remoção de acentos pela decomposição Unicode NFD (ç→c, ã→a); **(3) stopwords**: lista própria de 117 palavras funcionais do português; **(4) stemming**: `StemmerPT`, implementado do zero e inspirado nas etapas do RSLP (Orengo & Huyck, 2001): plural → feminino → advérbio → diminutivo → sufixo nominal → sufixo verbal → vogal final. Cada regra tem um tamanho mínimo de radical, para evitar cortes excessivos. Exemplos: *irrigação, irrigações, irrigar, irrigado* → `irrig`; *cultivo, cultivar, cultivada* → `cultiv`; *lagarta, lagartas* → `lagart`. Durante os testes, duas regras foram corrigidas: o aumentativo "-ão" cortava antes de "-ação" ("irrigação" virava "irrigac"), e "-ivo" separava "cultivo" de "cultivar". A aba 1 mostra o resultado de cada etapa (stopwords riscadas) e o tamanho do vocabulário nas quatro combinações: **47** termos sem stopwords nem stemming e **33** com os dois. Toda a redução vem das stopwords: neste corpus nenhuma palavra aparece em duas formas, então o stemming não encolhe o vocabulário. O ganho dele está em fazer as *consultas* casarem com os documentos (Figura 1).

3. Fase 2 — Índice invertido

`construir_indice()` gera um dicionário `termo → {doc: frequência}`, em ordem alfabética. A lista de documentos de cada termo dá o df(t), e a frequência alimenta o TF. A aba 2 exibe o índice em `st.dataframe` (termo, df, documentos, postings e IDF), com filtro, e em `st.json` (termo → [IDs]). Exemplo: `soj → ['Doc 1', 'Doc 2', 'Doc 4']`, `lagart → ['Doc 2', 'Doc 4']`.

4. Fase 3 — TF-IDF e ranqueamento

TF(t,d) = f(t,d) / |d|, com opção de TF bruto ou logarítmico (1 + $\log_{10}$ f); IDF(t) = $\log_{10}$(N / df(t)); TF-IDF = TF × IDF; score da consulta = Σ TF-IDF(t,d) sobre os termos distintos da consulta. A consulta passa pelo mesmo pipeline dos documentos. A aba 3 mostra a tabela ordenada do maior para o menor TF-IDF acumulado, com o **documento vencedor destacado**, o passo a passo (f, |d|, TF, df, N, IDF) e a matriz TF-IDF completa. Exemplo verificado à mão para "lagartas na soja" → `lagart, soj` ("na" é stopword):

Documento	|d|	TF-IDF(lagart) IDF = $\log_{10}$(5/2) = 0,3979	TF-IDF(soj) IDF = $\log_{10}$(5/3) = 0,2218	Score
Doc 2 (vencedor)	7	1/7 × 0,3979 = 0,0568	1/7 × 0,2218 = 0,0317	0,0885
Doc 4	8	1/8 × 0,3979 = 0,0497	1/8 × 0,2218 = 0,0277	0,0775
Doc 1	8	0	1/8 × 0,2218 = 0,0277	0,0277
Doc 3, Doc 5	7	0	0	0,0000

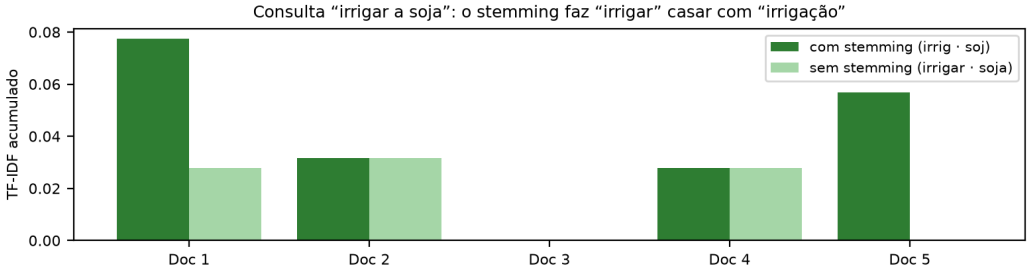


Figura 1 — Sem stemming, "irrigar" não existe no índice e só "soja" pontua (o Doc 2 vence por ser o mais curto); com stemming, o Doc 1, que fala de irrigação da soja, vence.

5. Bônus — Similaridade de cosseno

A consulta e cada documento viram vetores de pesos w = TF × IDF sobre o vocabulário, e cos(q,d) = (q·d) / (‖q‖·‖d‖). A aba 4 compara lado a lado o ranking por cosseno e o ranking por TF-IDF acumulado, com a norma ‖d‖ de cada documento. Para "lagartas na soja", o cosseno dá Doc 2 = 0,2799, Doc 4 = 0,2572 e Doc 1 = 0,0610, a mesma ordem do TF-IDF. Neste corpus de frases curtas e parecidas, o TF normalizado já compensa o tamanho, e os dois métodos quase sempre concordam. A diferença aparece com o **TF bruto**: para "soja", os Docs 1, 2 e 4 **empatam** no TF-IDF acumulado (0,2218 cada), e o cosseno desempata pelo

peso relativo do termo no documento (Doc 2 = 0,1363 contra Doc 1 = 0,1252). Em consultas de várias palavras, o cosseno premia documentos que cobrem vários termos na mesma proporção, em vez de somar pesos isolados.

6. Verificação

Os valores de TF-IDF e de cosseno do app foram comparados com um cálculo manual independente, e as diferenças ficaram abaixo de 10^{-12}. A interface foi testada automaticamente (Streamlit AppTest): consultas com e sem termos no vocabulário, as quatro combinações de stopwords/stemming, os três esquemas de TF e a aba de cosseno.

7. Divisão de tarefas da equipe

Integrante	Responsabilidades
Integrante 1	Fase 1: tokenização, normalização, lista de stopwords e implementação/ajuste do stemmer StemmerPT; aba de pré-processamento com as caixas de seleção.
Integrante 2	Fases 2 e 3: índice invertido, cálculo de TF/IDF/TF-IDF, ranqueamento com destaque do vencedor e verificação manual dos cálculos.
Integrante 3	Bônus e integração: similaridade de cosseno, interface Streamlit (abas, base editável), testes automatizados e redação do relatório.

Execução: `pip install -r requirements.txt` e `streamlit run agrosearch_app.py`.